

🎯 Placement ke liye sabse zyada priority

Must-do:

Largest → Second Largest → Reverse → Sorted → Frequency → Missing → Duplicate → Move Zeros → Pair Sum → Majority → Maximum Subarray → Stock Profit → Rotate → Kth Largest/Smallest

Ye **25 Arrays ka complete interview set** hai. Iske baad Array ke random 50–100 questions karne ki zarurat nahi hai; in 25 ko code karke practice karna zyada useful rahega.

Now react code dedo jo interviewe puch skte h likhne ke liye kya kya ho skta h
vaise

Haan. **React interview mein theory se zyada kabhi-kabhi “ye component/code likho”** bol dete hain. Fresher/service-based placement ke hisaab se ye coding tasks sabse important hain.

🔥 React Interview — Code Writing Questions

🟢 Level 1 — Must Do

1. Create a simple React component

Interviewer: “Create a functional component that displays Hello World.”

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

```
export default App;
```

2. Create a component using Props ★★

Interviewer: “Create a component that receives a name through props and displays it.”

```
function User({ name }) {  
  return <h2>Hello, {name}</h2>;  
}
```

```
function App() {  
  return <User name="Pranjal" />;  
}
```

```
export default App;
```

3. useState — Counter ★★★★★

Interviewer: “Create a counter with increment and decrement buttons.”

```
import { useState } from "react";
```

```
function App() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>{count}</h2>  
  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
  
      <button onClick={() => setCount(count - 1)}>  
        Decrement  
      </button>  
    </div>  
  );  
}
```

```
export default App;
```

4. Toggle Button ★★★★★

Interviewer: “Create a button that toggles between Show and Hide.”

```
import { useState } from "react";
```

```
function App() {
```

```

const [show, setShow] = useState(true);

return (
  <div>
    <button onClick={() => setShow(!show)}>
      {show ? "Hide" : "Show"}
    </button>

    {show && <p>Hello React</p>}
  </div>
);
}

export default App;

```

5. Input Field ★★

Interviewer: “Create an input field and display its value.”

```

import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <p>{name}</p>
    </div>
  );
}

export default App;

```

6. Controlled Form ★★

Interviewer: “Create a form with name and email fields and handle its submission.”

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    console.log(name);
    console.log(email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Name"
      />

      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />

      <button type="submit">Submit</button>
    </form>
  );
}

export default App;
```

Level 2 — Very Important

7. Render an Array using map() ★★☆☆

Interviewer: “Display a list of users using map().”

```
function App() {  
  const users = ["Aman", "Riya", "Rahul"];  
  
  return (  
    <div>  
      {users.map((user, index) => (  
        <p key={index}>{user}</p>  
      ))}  
    </div>  
  );  
}  
  
export default App;
```

8. Conditional Rendering ★★☆☆

Interviewer: “Show different content based on login status.”

```
function App() {  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h2>Welcome User</h2>  
      ) : (  
        <h2>Please Login</h2>  
      )}  
    </div>  
  );  
}  
  
export default App;
```

9. Search Filter ★★

Interviewer: “Create a search box that filters a list of users.”

```
import { useState } from "react";

function App() {
  const users = ["Aman", "Riya", "Rahul", "Priya"];
  const [search, setSearch] = useState("");

  const filteredUsers = users.filter((user) =>
    user.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <input
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        placeholder="Search"
      />

      {filteredUsers.map((user, index) => (
        <p key={index}>{user}</p>
      ))}
    </div>
  );
}

export default App;
```

10. Add Item to List ★★

Interviewer: “Create an input and add new items to a list.”

```
import { useState } from "react";

function App() {
  const [item, setItem] = useState("");
  const [items, setItems] = useState([]);

  function addItem() {
```

```

    if (item.trim() === "") return;

    setItems([...items, item]);
    setItem("");
  }

  return (
    <div>
      <input
        value={item}
        onChange={(e) => setItem(e.target.value)}
      />

      <button onClick={addItem}>Add</button>

      {items.map((item, index) => (
        <p key={index}>{item}</p>
      ))}
    </div>
  );
}

export default App;

```

11. Delete Item from List ★★

Interviewer: “Create a list where each item can be deleted.”

```

import { useState } from "react";

function App() {
  const [items, setItems] = useState(["Apple", "Banana", "Mango"]);

  function deleteItem(index) {
    setItems(items.filter((_, i) => i !== index));
  }

  return (
    <div>
      {items.map((item, index) => (
        <div key={index}>

```

```

        <span>{item}</span>

        <button onClick={() => deleteItem(index)}>
            Delete
        </button>
    </div>
    )}
</div>
);
}

export default App;

```

12. useEffect — Fetch API ★★★★★

Interviewer: “Fetch users from an API when the component mounts.”

```

import { useEffect, useState } from "react";

function App() {
    const [users, setUsers] = useState([]);

    useEffect(() => {
        fetch("https://jsonplaceholder.typicode.com/users")
            .then((response) => response.json())
            .then((data) => setUsers(data));
    }, []);

    return (
        <div>
            {users.map((user) => (
                <p key={user.id}>{user.name}</p>
            ))}
        </div>
    );
}

export default App;

```

13. Loading State ★★★★★

Interviewer: “Show Loading while API data is being fetched.”

```
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => response.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) {
    return <h2>Loading...</h2>;
  }

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default App;
```

14. useEffect Dependency ★★★★★

Interviewer: “Run an effect whenever count changes.”

```
import { useEffect, useState } from "react";

function App() {
  const [count, setCount] = useState(0);
```

```
useEffect(() => {
  console.log("Count changed:", count);
}, [count]);

return (
  <button onClick={() => setCount(count + 1)}>
    {count}
  </button>
);
}

export default App;
```

15. Parent to Child Data ★★★★★

Interviewer: “Pass data from parent component to child component.”

```
function Child({ name }) {
  return <h2>Hello {name}</h2>;
}

function App() {
  return <Child name="Pranjal" />;
}

export default App;
```

16. Child to Parent Data ★★★★★

Interviewer: “Send data from child component to parent.”

```
import { useState } from "react";

function Child({ sendData }) {
  return (
    <button onClick={() => sendData("Hello Parent")}>
      Send Data
    </button>
  );
}
```

```

}

function App() {
  const [message, setMessage] = useState("");

  return (
    <div>
      <Child sendData={setMessage} />
      <p>{message}</p>
    </div>
  );
}

export default App;

```

Level 3 — Interview mein kaafi useful

17. React Router — Two Pages ★★☆☆

Interviewer: “Create Home and About pages using React Router.”

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

```

```
    </Routes>
  </BrowserRouter>
);
}
```

```
export default App;
```

18. Fetch API with Axios ★★★★★

Interviewer: “Fetch data from an API using Axios.”

```
import axios from "axios";
import { useEffect, useState } from "react";
```

```
function App() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      });
  }, []);

  return (
    <div>
      {users.map((user) => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}
```

```
export default App;
```

19. Context API ★★★★★

Interviewer: “Create a context and access its value in a child component.”

```
import { createContext, useContext } from "react";

const UserContext = createContext();

function Child() {
  const name = useContext(UserContext);

  return <h2>{name}</h2>;
}

function App() {
  return (
    <UserContext.Provider value="Pranjal">
      <Child />
    </UserContext.Provider>
  );
}

export default App;
```

20. useRef ★★

Interviewer: “Focus an input using useRef.”

```
import { useRef } from "react";

function App() {
  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} />

      <button onClick={focusInput}>
        Focus
      </button>
    </div>
  );
}
```

```
);  
}
```

```
export default App;
```

★ Sabse Important 10

Agar interviewer tumhe **live React coding** de, to sabse pehle inhe perfect karo:

1. **Counter using useState**
2. **Toggle Show/Hide**
3. **Controlled Input**
4. **Form Handling**
5. **Array rendering using map**
6. **Search/Filter**
7. **Add & Delete items**
8. **API fetching using useEffect**
9. **Props — Parent → Child**
10. **Child → Parent using callback**

Aur tumhare **MERN fresher interview** ke liye uske baad **React Router + Axios + Context API + useRef** kar lena.

Ye list theory questions nahi hai — ye specifically woh type ke code-writing tasks hain jo interviewer tumhe editor/screen par likhne ko de sakta hai.

Node vgera ke bhi puchte h kya

Haan, **Node.js aur Express.js ka code bhi interview mein likhne ko de sakte hain**, especially MERN/full-stack fresher role mein.

React ke baad tumhe ye **most important code-writing tasks** karne chahiye:

🔥 Node.js

1. Simple HTTP server create karna
2. fs module se file read/write
3. File create/update/delete

4. path module use karna
5. Environment variables use karna
6. Async/await ka example
7. Promise handling
8. Error handling with try/catch

Express.js ★★

1. Basic Express server
2. GET API banana
3. POST API banana
4. PUT API banana
5. DELETE API banana
6. Route parameters — /users/:id
7. Query parameters — /users?name=...
8. Middleware banana
9. Error-handling middleware
10. JSON request body handle karna
11. CORS setup
12. Router banana
13. Controller + Routes structure
14. Authentication middleware
15. JWT generate/verify karna

MongoDB + Mongoose

1. Connect MongoDB
2. Create Schema/Model
3. Insert document
4. Find documents
5. Find by ID
6. Update document
7. Delete document
8. Search/filter
9. MongoDB queries
10. Mongoose relationships/populate

🎯 Tumhare placement ke liye order

React → Node → Express → MongoDB → Authentication/JWT → REST API

Aur haan, Node/Express mein bhi main tumhe wahi format dunga:

Interviewer question → simple code → output/response → Time/Space complexity (jahan meaningful ho).

React ke baad **Node.js ke 15 most important code-writing questions** se start karna best rahega.

Ha yrr dedo

Haan yrr 😊 **Node.js ke 15 most important code-writing questions** karte hain. Ye fresher/MERN interview ke liye kaafi relevant hain.

🔥 NODE.JS — 15 CODE WRITING QUESTIONS

1. Create a Basic HTTP Server ★★☆☆

Interviewer: “Create a simple HTTP server using Node.js.”

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.write("Hello Node.js");
  res.end();
});

server.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Output:

Server running on port 5000

Browser:

Hello Node.js

Time: $O(1)$ per request

Space: $O(1)$

2. Handle Different Routes ★★ ★

Interviewer: “Create different responses for different URLs.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  if (req.url === "/") {
    res.end("Home Page");
  }

  else if (req.url === "/about") {
    res.end("About Page");
  }

  else {
    res.end("Page Not Found");
  }
});

server.listen(5000);
```

Output:

/ → Home Page

/about → About Page

Time: $O(1)$

Space: $O(1)$

3. Read a File using fs ★★ ★

Interviewer: “Read a file using Node.js.”

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log(data);
});
```

Time: $O(n)$

Space: $O(n)$

4. Write Data to a File ★★

Interviewer: “Write data into a file using Node.js.”

```
const fs = require("fs");

fs.writeFile("data.txt", "Hello Node.js", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("File written successfully");
});
```

Output:

File written successfully

Time: $O(n)$

Space: $O(n)$

5. Append Data to a File ★★

Interviewer: “Add new data to an existing file.”

```
const fs = require("fs");

fs.appendFile("data.txt", "\nHello Again", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("Data added");
});
```

Time: $O(n)$

Space: $O(n)$

6. Delete a File ★★

Interviewer: “Delete a file using Node.js.”

```
const fs = require("fs");

fs.unlink("data.txt", (err) => {

  if (err) {
    console.log(err);
    return;
  }

  console.log("File deleted");
});
```

Time: $O(n)$

Space: $O(1)$

7. Use Async/Await

Interviewer: “Read a file using async/await.”

```
const fs = require("fs").promises;

async function readFile() {
  try {
    const data = await fs.readFile("data.txt", "utf8");

    console.log(data);
  } catch (error) {
    console.log(error);
  }
}

readFile();
```

Time: O(n)

Space: O(n)

8. Handle Errors using try/catch

Interviewer: “Handle errors in an asynchronous function.”

```
async function getData() {

  try {
    let data = await fetchData();

    console.log(data);
  } catch (error) {
    console.log("Something went wrong");
  }
}

getData();
```

Main point: try mein successful code, catch mein error handling.

9. Create a Simple REST API ★★

Interviewer: “Create a GET API using Node.js.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.setHeader("Content-Type", "application/json");

  if (req.url === "/users") {

    const users = [
      { id: 1, name: "Aman" },
      { id: 2, name: "Riya" }
    ];

    res.end(JSON.stringify(users));
  }
});

server.listen(5000);
```

Response:

```
[
  {
    "id": 1,
    "name": "Aman"
  },
  {
    "id": 2,
    "name": "Riya"
  }
]
```

10. Use Environment Variables ★★

Interviewer: “How do you use environment variables in Node.js?”

.env

```
PORT=5000
DB_URL=mongodb://localhost:27017/mydb
```

server.js

```
require("dotenv").config();
```

```
const port = process.env.PORT;
```

```
console.log(port);
```

Output:

5000

👉 Real projects mein **database URL**, **API keys**, **JWT secret** etc. environment variables mein rakhte hain.

11. Create a Promise ★★

Interviewer: “Create and handle a Promise.”

```
function getData() {
  return new Promise((resolve, reject) => {

    let success = true;

    if (success) {
      resolve("Data received");
    } else {
      reject("Something went wrong");
    }
  });
}
```

```
getData()
  .then((data) => console.log(data))
  .catch((error) => console.log(error));
```

Output:

Data received

12. Use EventEmitter ★★

Interviewer: “Create a custom event using EventEmitter.”

```
const EventEmitter = require("events");
```

```
const event = new EventEmitter();
```

```
event.on("message", () => {  
  console.log("Message received");  
});
```

```
event.emit("message");
```

Output:

Message received

Important: on() listener register karta hai, emit() event trigger karta hai.

13. Create a Custom Module ★★

Interviewer: “Create and import a custom Node.js module.”

math.js

```
function add(a, b) {  
  return a + b;  
}
```

```
module.exports = add;
```

app.js

```
const add = require("./math");
```

```
console.log(add(10, 20));
```

Output:

30

14. Get Data from URL ★★

Interviewer: “Get query parameters from a URL.”

```
const http = require("http");
const url = require("url");

const server = http.createServer((req, res) => {

  const query = url.parse(req.url, true).query;

  res.end(`Hello ${query.name}`);
});

server.listen(5000);
```

URL:

http://localhost:5000/?name=Pranjal

Output:

Hello Pranjal

15. Create a Server with JSON Response ★★

Interviewer: “Return JSON data from a Node.js server.”

```
const http = require("http");

const server = http.createServer((req, res) => {

  res.setHeader("Content-Type", "application/json");

  const user = {
    name: "Pranjal",
    role: "Developer"
  };
});
```

```
res.end(JSON.stringify(user));  
});
```

```
server.listen(5000);
```

Response:

```
{  
  "name": "Pranjal",  
  "role": "Developer"  
}
```

Time: O(1)

Space: O(1)

Node.js mein sabse important

Agar **15 mein se priority** nikalni ho:

★ ★ ★ MUST DO

1. Basic HTTP Server
2. Routes
3. fs read/write
4. Async/Await
5. Error Handling
6. REST API
7. Environment Variables
8. Promise
9. JSON Response
10. URL/Query Parameters

★ ★ Secondary

11. Append/Delete file
 12. EventEmitter
 13. Custom Module
-

But ek important point

MERN interview mein Node.js se zyada Express.js ke code-writing questions aane ke chances hote hain, kyunki actual backend APIs generally Express se banate ho.

Isliye next **Express.js ke 15 most important live-coding questions** karna sabse useful rahega:

GET → POST → PUT → DELETE → params → query → middleware → error middleware → router → controller → JWT auth → protected route → MongoDB API.

Ok start aur ha mai esmodules use krta hu naki common js

Haan yrr, **tum ES Modules use karti ho**, so ab se Node/Express ka code **import/export syntax** mein hi dunga — `require()` / `module.exports` nahi.

Chalo **Express.js ke 15 most important code-writing questions** start karte hain, same interview format mein.

EXPRESS.JS — COMPLETE CODE-WRITING LECTURE

1. Create a Basic Express Server ★★ ★

Interviewer: “Create a basic Express server.”

```
import express from "express";

const app = express();

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Output:

Server running on port 5000

Time: O(1)
Space: O(1)

2. Create a GET API ★★

Interviewer: “Create a GET API that returns a message.”

```
import express from "express";

const app = express();

app.get("/api/users", (req, res) => {
  res.json({
    message: "Users fetched successfully"
  });
});

app.listen(5000);
```

Response:

```
{
  "message": "Users fetched successfully"
}
```

3. Create a POST API ★★

Interviewer: “Create a POST API to receive user data.”

```
import express from "express";

const app = express();

app.use(express.json());

app.post("/api/users", (req, res) => {
  const { name, email } = req.body;

  res.json({
    message: "User created",
```

```
    name,  
    email  
  });  
};  
  
app.listen(5000);
```

Request Body:

```
{  
  "name": "Pranjal",  
  "email": "pranjal@gmail.com"  
}
```

Response:

```
{  
  "message": "User created",  
  "name": "Pranjal",  
  "email": "pranjal@gmail.com"  
}
```

4. Create PUT API ★★

Interviewer: “Create an API to update a user.”

```
app.put("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  const { name } = req.body;  
  
  res.json({  
    message: "User updated",  
    id,  
    name  
  });  
});
```

URL:

PUT /api/users/10

5. Create DELETE API ★★

Interviewer: “Create an API to delete a user.”

```
app.delete("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  
  res.json({  
    message: "User deleted",  
    id  
  });  
});
```

URL:

DELETE /api/users/10

6. Route Parameters ★★

Interviewer: “Get an ID from the URL.”

```
app.get("/api/users/:id", (req, res) => {  
  const id = req.params.id;  
  
  res.json({  
    userId: id  
  });  
});
```

URL:

/api/users/25

Response:

```
{  
  "userId": "25"  
}
```

7. Query Parameters

Interviewer: “Get search data from query parameters.”

```
app.get("/api/users", (req, res) => {  
  const name = req.query.name;  
  
  res.json({  
    search: name  
  });  
});
```

URL:

/api/users?name=Pranjal

Response:

```
{  
  "search": "Pranjal"  
}
```

Yaad rakho:

```
/users/:id    → req.params.id  
/users?name=x → req.query.name
```

8. Middleware

Interviewer: “Create a middleware that logs every request.”

```
function logger(req, res, next) {  
  console.log(req.method, req.url);  
  next();  
}
```

```
app.use(logger);
```

next() ka kaam hai **request ko next middleware/route par bhejna.**

9. Authentication Middleware ★★

Interviewer: “Create middleware to check whether a user is authenticated.”

```
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization;  
  
  if (!token) {  
    return res.status(401).json({  
      message: "Unauthorized"  
    });  
  }  
  
  next();  
}
```

Protected route:

```
app.get("/api/profile", authMiddleware, (req, res) => {  
  res.json({  
    message: "Profile data"  
  });  
});
```

10. Error Handling Middleware ★★

Interviewer: “Create an error-handling middleware.”

```
app.use((err, req, res, next) => {  
  console.log(err);  
  
  res.status(500).json({  
    message: "Something went wrong"  
  });  
});
```

⚠ Express error middleware mein **4 parameters** hote hain:

```
(err, req, res, next)
```

11. Router ★★

Interviewer: “Create separate routes using Express Router.”

routes/userRoutes.js

```
import express from "express";
```

```
const router = express.Router();
```

```
router.get("/", (req, res) => {  
  res.json({  
    message: "All users"  
  });  
});
```

```
router.post("/", (req, res) => {  
  res.json({  
    message: "User created"  
  });  
});
```

```
export default router;
```

server.js

```
import express from "express";
```

```
import userRoutes from "../routes/userRoutes.js";
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use("/api/users", userRoutes);
```

```
app.listen(5000);
```

12. Controller + Route ★★

Interviewer: “Separate controller logic from routes.”

controllers/userController.js

```
export function getUsers(req, res) {  
  res.json({  
    message: "All users"  
  });  
}
```

routes/userRoutes.js

```
import express from "express";  
import { getUsers } from "../controllers/userController.js";
```

```
const router = express.Router();
```

```
router.get("/", getUsers);
```

```
export default router;
```

server.js

```
import express from "express";  
import userRoutes from "../routes/userRoutes.js";
```

```
const app = express();
```

```
app.use("/api/users", userRoutes);
```

```
app.listen(5000);
```

13. Async Controller with Error Handling



Interviewer: "Create an asynchronous controller."

```
export async function getUsers(req, res) {  
  try {  
    const users = await User.find();  
  
    res.json(users);  
  } catch (error) {  
    res.status(500).json({
```

```
    message: "Failed to fetch users"
  });
}
```

Ye **MongoDB + Mongoose** ke saath bahut common hai.

14. JWT Authentication ★★

Interviewer: “Generate a JWT token after login.”

```
import jwt from "jsonwebtoken";
```

```
const token = jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);
```

```
res.json({
  token
});
```

Verify token middleware:

```
import jwt from "jsonwebtoken";
```

```
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];

  if (!token) {
    return res.status(401).json({
      message: "Unauthorized"
    });
  }

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );
```

```
    req.user = decoded;

    next();
  } catch (error) {
    res.status(401).json({
      message: "Invalid token"
    });
  }
}

export default authMiddleware;
```

15. Complete CRUD API ★★

Interviewer: “Create CRUD APIs for users.”

```
import express from "express";

const app = express();

app.use(express.json());

let users = [];

app.get("/users", (req, res) => {
  res.json(users);
});

app.post("/users", (req, res) => {
  const user = {
    id: users.length + 1,
    name: req.body.name
  };

  users.push(user);

  res.status(201).json(user);
});
```

```
app.put("/users/:id", (req, res) => {
  const user = users.find(
    user => user.id === Number(req.params.id)
  );

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  user.name = req.body.name;

  res.json(user);
});

app.delete("/users/:id", (req, res) => {
  users = users.filter(
    user => user.id !== Number(req.params.id)
  );

  res.json({
    message: "User deleted"
  });
});

app.listen(5000);
```

EXPRESS — Placement Priority

★★★★ Must Practice

1. Basic Express Server
2. GET API
3. POST API
4. PUT API
5. DELETE API
6. req.body
7. req.params

8. req.query
9. Middleware
10. Authentication Middleware
11. Router
12. Controller
13. Error Handling
14. JWT Authentication
15. CRUD API

Tumhare MERN interviews ke liye ye Express coding set kaafi strong hai. Iske baad **MongoDB + Mongoose ke code-writing questions** karna logical next step hai.

Ha start

Bilkul yrr 🤪 **MongoDB + Mongoose code-writing interview prep** start karte hain.

Same pattern rakhenge: **Question → Built-in/simple → Without built-in → Optimized → Complexity.**

Aur Node/Express ki tarah **ES Modules** use karenge.

MongoDB + Mongoose — Most Important Code Questions

1. Connect MongoDB using Mongoose ★★★★★

Interviewer: “Write code to connect a Node.js application with MongoDB using Mongoose.”

```
import mongoose from "mongoose";

mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((error) => {
    console.log(error);
  });
```

Time: O(1) application-side

Space: O(1)

2. Create a Mongoose Schema ★★★★★

Interviewer: “Create a User schema with name, email and password.”

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});

export default mongoose.model("User", userSchema);
```

Time: O(1)

Space: O(1)

3. Insert/Create a Document ★★★★★

Interviewer: “Write an API to create a new user.”

```
app.post("/users", async (req, res) => {
  const { name, email, password } = req.body;

  const user = await User.create({
    name,
    email,
    password
  });

  res.json(user);
});
```

Time: O(1) average for one document

Space: O(1)

4. Find All Users ★★★★★

Interviewer: “Write a query to fetch all users.”

```
const users = await User.find();
```

```
console.log(users);
```

Time: $O(n)$

Space: $O(n)$ for returned documents

5. Find User by ID ★★

Interviewer: “Find a user using their ID.”

```
const user = await User.findById(req.params.id);
```

```
console.log(user);
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

6. Find User by Email ★★

Interviewer: “Find a user by email.”

```
const user = await User.findOne({  
  email: "abc@gmail.com"  
});
```

```
console.log(user);
```

Time: $O(n)$ without index, approximately $O(1)$ with an index

Space: $O(1)$

7. Update User ★★

Interviewer: “Update a user’s name using their ID.”

```
const user = await User.findByIdAndUpdate(  
  req.params.id,  
  { name: "Pranjal" },  
  { new: true }  
);
```

```
console.log(user);
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

8. Delete User ★★★★★

Interviewer: “Delete a user using their ID.”

```
await User.findByIdAndDelete(req.params.id);
```

```
console.log("User deleted");
```

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

9. Find Users with a Condition ★★★★★

Interviewer: “Find users whose age is greater than 18.”

```
const users = await User.find({  
  age: { $gt: 18 }  
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable index

Space: $O(n)$

10. Projection ★★★★★

Interviewer: “Fetch only name and email, not password.”

```
const users = await User.find()  
  .select("name email");
```

```
console.log(users);
```

Time: $O(n)$ without suitable index
Space: $O(n)$

11. Sort Documents ★★

Interviewer: “Fetch users sorted by name.”

```
const users = await User.find()  
  .sort({ name: 1 });
```

```
console.log(users);
```

1 → ascending

-1 → descending

Time: depends on index; without suitable index, sorting can be $O(n \log n)$

Space: depends on query execution

12. Limit Documents ★★

Interviewer: “Fetch only 5 users.”

```
const users = await User.find()  
  .limit(5);
```

```
console.log(users);
```

Time: depends on query/index

Space: $O(5)$ returned result

13. Skip Documents ★★

Interviewer: “Skip the first 5 users.”

```
const users = await User.find()  
  .skip(5)  
  .limit(5);
```

```
console.log(users);
```

Time: depends on database/index
Space: $O(5)$

14. Count Documents ★★

Interviewer: “Count the number of users.”

```
const count = await User.countDocuments();
```

```
console.log(count);
```

Time: depends on query/index
Space: $O(1)$

15. Check Whether User Exists ★★★

Interviewer: “Check whether a user with a particular email exists.”

```
const user = await User.findOne({  
  email: "abc@gmail.com"  
});
```

```
if (user) {  
  console.log("User exists");  
} else {  
  console.log("User does not exist");  
}
```

Time: $O(n)$ without index, approximately $O(1)$ with email index
Space: $O(1)$

🔥 Placement ke liye sabse important

In MongoDB/Mongoose me **ye pakka strong karo:**

1. find()
2. findOne()
3. findById()
4. create()

5. findByIdAndUpdate()
6. findByIdAndDelete()
7. \$gt, \$lt, \$gte, \$lte
8. \$in, \$ne
9. Projection — .select()
- 10..sort()
- 11..limit() / .skip()
- 12.countDocuments()
- 13.Schema + Model
- 14.MongoDB connection

Next: \$in, \$ne, \$and, \$or, pagination, populate(), aggregation aur proper CRUD API — ye interview ke liye next important set hai.

Ha dedo

Chalo yrr 🔥 **MongoDB/Mongoose ka next important set** — simple interview-style code ke saath.

16. \$in Operator ★★ ★

Interviewer: “Find users whose age is either 18, 20 or 22.”

```
const users = await User.find({  
  age: { $in: [18, 20, 22] }  
});
```

```
console.log(users);
```

Time: O(n) without suitable index

Space: O(n)

17. \$ne Operator ★★

Interviewer: “Find users whose role is not admin.”

```
const users = await User.find({  
  role: { $ne: "admin" }  
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable index

Space: $O(n)$

18. \$and Operator ★★★★★

Interviewer: “Find users whose age is greater than 18 and role is user.”

```
const users = await User.find({
  $and: [
    { age: { $gt: 18 } },
    { role: "user" }
  ]
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable indexes

Space: $O(n)$

19. \$or Operator ★★★★★

Interviewer: “Find users whose role is admin or manager.”

```
const users = await User.find({
  $or: [
    { role: "admin" },
    { role: "manager" }
  ]
});
```

```
console.log(users);
```

Time: $O(n)$ without suitable indexes

Space: $O(n)$

20. Pagination ★★☆☆

Interviewer: “Implement pagination and return 5 users per page.”

```
const page = 2;
const limit = 5;

const users = await User.find()
  .skip((page - 1) * limit)
  .limit(limit);

console.log(users);
```

For page 2:

$\text{skip} = (2 - 1) \times 5 = 5$

So first 5 users skip honge.

Time: depends on database/index and skip size

Space: $O(\text{limit})$

21. populate() ★★☆☆

Interviewer: “Fetch bookings along with the user details.”

Suppose Booking schema:

```
const bookingSchema = new mongoose.Schema({
  car: String,
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});
```

Query:

```
const bookings = await Booking.find()
  .populate("user");

console.log(bookings);
```

Time: depends on number of documents and related lookups
Space: O(n)

22. Populate Specific Fields ★★ ★

Interviewer: “Populate only the user’s name and email.”

```
const bookings = await Booking.find()  
  .populate("user", "name email");
```

```
console.log(bookings);
```

Password jaise unnecessary fields nahi aayenge.

Time: depends on query

Space: O(n)

23. Aggregation — \$match ★★ ★

Interviewer: “Find users whose age is greater than 18 using aggregation.”

```
const users = await User.aggregate([  
  {  
    $match: {  
      age: { $gt: 18 }  
    }  
  }  
]);
```

```
console.log(users);
```

Time: depends on pipeline/index

Space: depends on result/pipeline

24. Aggregation — \$group ★★ ★

Interviewer: “Count users according to their role.”

```
const result = await User.aggregate([
  {
    $group: {
      _id: "$role",
      count: { $sum: 1 }
    }
  }
]);
```

```
console.log(result);
```

Example output:

```
[
  { _id: "user", count: 10 },
  { _id: "admin", count: 2 }
]
```

Time: $O(n)$ without useful optimization

Space: depends on number of groups

25. Aggregation — \$sort ★★

Interviewer: “Sort users by age in descending order.”

```
const users = await User.aggregate([
  {
    $sort: {
      age: -1
    }
  }
]);
```

```
console.log(users);
```

1 → ascending

-1 → descending

Time: generally $O(n \log n)$ without a suitable index

Space: depends on execution

26. Complete CRUD API ★★

Ye **bahut important** hai because interviewer bol sakta hai:

“Create a basic CRUD API using Express and MongoDB.”

Create

```
app.post("/users", async (req, res) => {  
  const user = await User.create(req.body);  
  
  res.json(user);  
});
```

Read

```
app.get("/users", async (req, res) => {  
  const users = await User.find();  
  
  res.json(users);  
});
```

Update

```
app.put("/users/:id", async (req, res) => {  
  const user = await User.findByIdAndUpdate(  
    req.params.id,  
    req.body,  
    { new: true }  
  );  
  
  res.json(user);  
});
```

Delete

```
app.delete("/users/:id", async (req, res) => {  
  await User.findByIdAndDelete(req.params.id);  
  
  res.json({  
    message: "User deleted"  
  });  
});
```

Time: CRUD operation ke according vary karega

Space: returned data ke according

MongoDB/Mongoose Placement Priority

Agar time kam ho to **is order mein karo**:

★★★ Must Do

- Schema + Model
- create()
- find()
- findOne()
- findById()
- Update
- Delete
- \$gt, \$lt, \$gte, \$lte
- \$in, \$ne
- \$and, \$or
- Projection
- populate()
- Pagination
- Complete CRUD

★★ Good to Know

- Aggregation
- \$match
- \$group
- \$sort
- Indexing

Ab MongoDB ka **basic + interview code portion kaafi strong** ho gaya. Next logically **Authentication + JWT code-writing** karna chahiye — register, login, bcrypt password hashing, JWT generate, auth middleware aur protected route.

Ha krade

Chalo yrr  ab **Authentication + JWT** karte hain. Ye MERN fresher interview mein **bahut important code-writing topic** hai.

Same pattern: **interviewer question** → **simple code** → **complexity**.
Aur haan, **ES Modules (import/export)** hi use karenge.

Authentication + JWT — Interview Code

1. Hash Password using bcrypt ★★★★★

Interviewer: “Hash the user’s password before storing it.”

```
import bcrypt from "bcrypt";

const hashedPassword = await bcrypt.hash(password, 10);

console.log(hashedPassword);
```

Time: $O(1)$ practically depends on bcrypt cost

Space: $O(1)$

2. Compare Password ★★★★★

Interviewer: “Check whether the entered password matches the hashed password.”

```
import bcrypt from "bcrypt";

const isMatch = await bcrypt.compare(
  password,
  user.password
);

console.log(isMatch);
```

Output:

true

or

false

Time: Depends on bcrypt cost

Space: $O(1)$

3. Register User ★★

Interviewer: “Create a registration API with password hashing.”

```
import bcrypt from "bcrypt";

app.post("/register", async (req, res) => {
  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = await User.create({
    name,
    email,
    password: hashedPassword
  });

  res.status(201).json({
    message: "User registered",
    user
  });
});
```

Time: Depends on bcrypt + database operation

Space: O(1)

4. Login User ★★

Interviewer: “Create a login API that verifies email and password.”

```
import bcrypt from "bcrypt";

app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
```

```

    return res.status(404).json({
      message: "User not found"
    });
  }

  const isMatch = await bcrypt.compare(
    password,
    user.password
  );

  if (!isMatch) {
    return res.status(401).json({
      message: "Invalid password"
    });
  }

  res.json({
    message: "Login successful"
  });
});

```

Time: Depends on DB + bcrypt

Space: O(1)

5. Generate JWT Token ★★

Interviewer: “Generate a JWT token after successful login.”

```
import jwt from "jsonwebtoken";
```

```

const token = jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1d" }
);

```

```
console.log(token);
```

Time: O(1) for practical purposes

Space: O(1)

6. Login + JWT ★★

Ye **bahut important combined question** hai.

Interviewer: “Create a login API that returns a JWT token.”

```
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";

app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
    return res.status(404).json({
      message: "User not found"
    });
  }

  const isMatch = await bcrypt.compare(
    password,
    user.password
  );

  if (!isMatch) {
    return res.status(401).json({
      message: "Invalid password"
    });
  }

  const token = jwt.sign(
    { userId: user._id },
    process.env.JWT_SECRET,
    { expiresIn: "1d" }
  );

  res.json({
    message: "Login successful",
    token
  });
});
```

```
});  
});
```

Time: DB + bcrypt + JWT generation

Space: O(1)

7. Verify JWT Token ★★

Interviewer: “Verify a JWT token.”

```
import jwt from "jsonwebtoken";
```

```
const decoded = jwt.verify(  
  token,  
  process.env.JWT_SECRET  
);
```

```
console.log(decoded);
```

Time: O(1)

Space: O(1)

8. Authentication Middleware ★★

Interviewer: “Create middleware to protect routes using JWT.”

```
import jwt from "jsonwebtoken";
```

```
function authMiddleware(req, res, next) {  
  const token = req.headers.authorization?.split(" ")[1];  
  
  if (!token) {  
    return res.status(401).json({  
      message: "Unauthorized"  
    });  
  }  
}
```

```
try {  
  const decoded = jwt.verify(  
    token,  
    process.env.JWT_SECRET  
  );  
  next();  
}
```

```

    token,
    process.env.JWT_SECRET
  );

  req.user = decoded;

  next();
} catch (error) {
  res.status(401).json({
    message: "Invalid token"
  });
}
}

export default authMiddleware;

```

Time: O(1)

Space: O(1)

9. Protected Route ★★

Interviewer: “Create a protected profile route.”

```

app.get(
  "/profile",
  authMiddleware,
  async (req, res) => {

    const user = await User.findById(
      req.user.userId
    );

    res.json(user);
  }
);

```

Yahan:

authMiddleware

pehle token verify karega.

Then:

```
req.user.userId
```

se user fetch hoga.

Time: $O(1)$ average with indexed `_id`

Space: $O(1)$

10. Check Duplicate Email ★★

Interviewer: “Prevent registration if email already exists.”

```
const existingUser = await User.findOne({
  email
});

if (existingUser) {
  return res.status(400).json({
    message: "Email already exists"
  });
}
```

Then user create karo.

Time: $O(n)$ without index, approximately $O(1)$ with email index

Space: $O(1)$

11. Logout ★★

JWT ka simple logout usually client-side token removal hota hai.

Interviewer: “Implement logout.”

```
app.post("/logout", (req, res) => {
  res.json({
    message: "Logout successful"
  });
});
```

Agar JWT **httpOnly cookie** mein stored hai, then cookie clear kar sakte ho:

```
res.clearCookie("token");

res.json({
  message: "Logout successful"
});
```

12. Role-Based Authorization ★★

Interviewer: “Allow only admins to access this route.”

```
function adminOnly(req, res, next) {
  if (req.user.role !== "admin") {
    return res.status(403).json({
      message: "Access denied"
    });
  }

  next();
}
```

Route:

```
app.get(
  "/admin",
  authMiddleware,
  adminOnly,
  (req, res) => {
    res.json({
      message: "Welcome Admin"
    });
  }
);
```

Time: O(1)

Space: O(1)

Authentication mein ye MUST DO

Placement ke liye sabse pehle ye **8 codes** perfectly kar lo:

1. ★★★★★ Register API
2. ★★★★★ Login API
3. ★★★★★ bcrypt hash
4. ★★★★★ bcrypt compare
5. ★★★★★ JWT generate
6. ★★★★★ JWT verify
7. ★★★★★ Auth middleware
8. ★★★★★ Protected route

Uske baad:

9. Duplicate email
10. Role-based authorization
11. Logout
12. Cookies

Next logical step: Complete MERN CRUD API — Model → Controller → Routes → Express server → MongoDB → JWT protected CRUD. Ye ekdum **real interview coding task** jaisa hoga.

Ha bta jldi

Haan yrr, **ab Complete MERN CRUD API** karte hain — ye interview mein sabse useful practical task hai.

Complete MERN CRUD — Interview Task

Interviewer:

> “Create a CRUD API using Node.js, Express, MongoDB and Mongoose.”

1. User Model

```
import mongoose from "mongoose";
```

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

export default mongoose.model("User", userSchema);
```

2. Create User — POST ★★★★★

```
app.post("/users", async (req, res) => {
  try {
    const user = await User.create(req.body);

    res.status(201).json(user);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: $O(1)$ average

Space: $O(1)$

3. Get All Users — GET ★★★★★

```
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();

    res.json(users);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: $O(n)$

Space: $O(n)$

4. Get Single User — GET ★★★★★

```
app.get("/users/:id", async (req, res) => {
  try {
    const user = await User.findById(req.params.id);

    res.json(user);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: O(1) average with `_id` index

Space: O(1)

5. Update User — PUT ★★★★★

```
app.put("/users/:id", async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    res.json(user);
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: O(1) average with `_id` index

Space: O(1)

6. Delete User — DELETE ★★

```
app.delete("/users/:id", async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);

    res.json({
      message: "User deleted"
    });
  } catch (error) {
    res.status(500).json({
      message: "Server error"
    });
  }
});
```

Time: O(1) average with `_id` index

Space: O(1)

★ Protected CRUD

Interviewer agar bole:

“Make the CRUD API accessible only to logged-in users.”

Bas middleware add:

```
app.get("/users", authMiddleware, async (req, res) => {
  const users = await User.find();

  res.json(users);
});
```

Same way POST, PUT, DELETE mein bhi:

```
app.post("/users", authMiddleware, ...);
app.put("/users/:id", authMiddleware, ...);
app.delete("/users/:id", authMiddleware, ...);
```
